

Tensor4all stack: development update

tenferro-rs · tensor4all-rs · hataori-rs · bindings

Hiroshi Shinaoka

tensor4all meeting, 2026-09-22

With S. Terasaki, R. Watanabe, L. Cheng, M. K. Ritter (ACI / TCI algorithms), S. Badr, K. Inayoshi, S. Dirnböck, M. Frankenbach, N. Ritz, J.-G. Liu, Y. Zhao and the HKUST(GZ) group

The community is growing

34

contributors, whole org
29 in May

12

contributors, Rust stack
7 in May

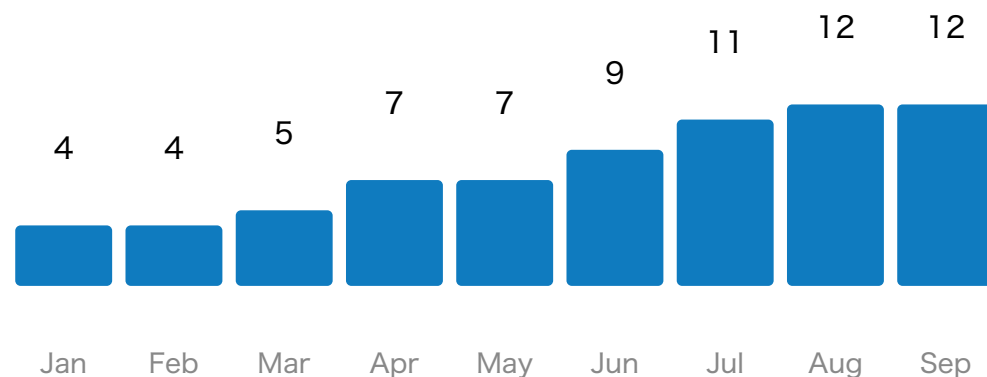
78

stars on tenferro-rs
5 in May

9

external repos using
tenferro
0 in May

Cumulative contributors, Rust stack (2026)



- **tenferro-rs**: committers 3 → 6 since June; five new issue reporters from Jin-Guo Liu's group
- **tensor4all-rs**: Lingrui Cheng landed 14 PRs; Samuel Badr, Ken Inayoshi joined
- Dependents: yao-rs (Liu), TeNeT (Watanabe), latticeqcd-rs, hataori-rs

Distinct commit authors since 2024 on GitHub, bots and forked repositories excluded; org total covers Julia, C++ and Rust repositories. Stars and dependents from the GitHub API on 2026-09-22.

Who contributed in 2026

Rust stack (joined since May, ordered by first commit)

Name	Affiliation	Repositories
Hiroshi Shinaoka	Saitama	all
Satoshi Terasaki	AtelierArith	tenferro-rs, tenferro-benchmark, strided-rs, hataori-rs
Jin-Guo Liu	HKUST(GZ)	omeinsum-rs, tenferro-rs
Ken Inayoshi	Saitama	tensor4all-rs
Nepomuk Ritz	LMU	tensor4all-rs
Selina Dirnböck	TU Wien	tensor4all-rs
▪ Xiwei Pan	HKUST(GZ)	omeinsum-rs
▪ Ryo Watanabe	Osaka	tenferro-rs, strided-rs
▪ Yusheng Zhao	HKUST(GZ)	omeinsum-rs, tenferro-rs
▪ Samuel Badr	TU Wien	tensor4all-rs
▪ Lingrui Cheng	LMU / TUM	tensor4all-rs, tensor4all-benchmark

Julia and C++ libraries

Name	Repositories
Marc K. Ritter	TensorCrossInterpolation.jl, AlternatingCrossInterpolation.jl
Hirone Ishida	tensorizingflows
Martin Mikkelsen	InterpolativeQTT.jl, Tensor4all.jl
Nepomuk Ritz	Tensor4all.jl
Samuel Badr	QuanticsGrids.jl, QuanticsTCI.jl
Markus Frankenbach	AlternatingCrossInterpolation.jl, TCIAgorithms.jl
T. Kloss, Y. Núñez Fernández, S. B., J. Fowkes	xfac

Issue reporters without commits: Guo P. Chen, LeoXia (HKUST(GZ)).

The stack, and what this talk covers

Layer	Repository and role
Bindings	Tensor4all.jl (Julia via C API) · tensor4all-py (PyO3 prototype)
Tensor networks	tensor4all-rs : TreeTN, TCI, ACI, quantics, partitioned networks
Dense tensor engine	tenferro-rs : tensors, sessions, einsum, linalg, FFT, autodiff; CPU / CUDA / WebGPU / Metal
Parallel runtime	hataori-rs : dynamically scheduled <code>pmap</code> over threads and MPI ranks

- Period: mid-May to mid-September 2026. Full features live in Rust; Julia and Python are thin entry points.
- Plan: community → tenferro-rs (15 min) → tensor4all-rs (20 min) → hataori-rs, bindings, next steps (15 min).

Four months at a glance

Repository	Merged PRs	Releases
tenferro-rs	about 380	v0.2 (Jul) → v0.3 → v0.4 → v0.5 → v0.6 (Sep 21) , on crates.io
tensor4all-rs	about 115	v0.2.0 (Jun)
strided-rs	about 65	v0.3 (Jul) → v0.4 (Aug), on crates.io
hataori-rs	new	first working engine (Aug)
Tensor4all.jl	10	Windows CI

- Every release ships migration notes; GPU CI runs on RunPod.
- Review, not typing, is the bottleneck: most code is written by agents under human gates (last slides).

tenferro-rs: what changed since May

tenferro-rs

- **Execution:** one explicit backend session shared by eager, autodiff and traced paths. Building the backend once removed most of the small-tensor overhead gap to PyTorch.
- **Linear algebra:** full SVD on CPU and CUDA, incremental Householder QR, rank-revealing QR; derivative rules generated from linearization.
- **Einsum:** strided-rs replay plans, ellipsis everywhere, TBLIS and grouped GEMM providers, a tropical (semiring) extension.
- **CUDA:** cached cuTENSOR plans, cuFFT, cuSOLVER driver selection, asynchronous event contract. Unsupported ops return typed errors, never a silent CPU fallback.
- **v0.6 (Sep 21):** open scalar contract, so downstream crates can define their own scalar types. WebGPU and Metal remain experimental.

Benchmark on NVIDIA A100

tenferro-benchmark

Dense linear algebra, f64 (ms)

Problem	tenferro	PyTorch
matmul 3072	3.30	3.12
batched matmul	2.32	2.08
eigh 1024	14.0	14.3
QR 1536	12.0	11.8
SVD 256	12.2	12.1
solve 2048	10.1	9.1

Permutation, 2 GiB tensors (ms)

Pattern	tenferro	cuTENSOR	Torch
18D reverse	8.8	8.7	33.2
23D reverse	7.7	7.6	45.7
2D transpose	5.7	5.5	20.8
3D transpose	5.4	5.3	27.4
24D TN-shaped	5.6	5.5	5.3

- Dense linalg and GEMM within a few percent of PyTorch and of cuBLASLt / cuSOLVER: no backend overhead.
- High-rank permutations run at cuTENSOR speed, up to 6× faster than PyTorch.

- **New algorithms:** randomized TreeTN contraction, cross interpolation native to trees, partitioned tree networks with reconstruction.
- **Parallel:** PartitionedTT adaptive interpolation runs on hataori-rs, Rayon by default and MPI opt-in.
- **Quality:** an August audit removed library panics, introduced typed errors, layering rules and a performance evidence ledger.
- **Boundaries:** C API for Julia unchanged; a PyO3 crate for Python started.

TreeTN: ground states, dynamics, storage tensor4all-rs

- Two-site DMRG, TDVP and GSE-TDVP on general trees; data structures and sweep plans follow ITensorNetworks.jl.
- **Root-edge-first sweep ordering:** fixed a ten-orders-of-magnitude accuracy loss on branching trees.
- Variational fitting of sums of TreeTNs; dense tensor $\rightarrow$ TT-SVD constructor.
- HDF5 schema for TreeTNs and an explicit CUDA contraction path.

Successive randomized compression (SRC)

tensor4all-rs

- Compresses tensor-network products without forming the exact high-rank product: Gaussian sketches plus local QR and projection. The MPO $\times$ MPS method of Camaño, Epperly and Tropp is extended here to general trees.
- Rust supports fixed-rank and adaptive variants; the C API currently exposes fixed-rank SRC. Adaptive SRC uses tenferro-backed QR / RRQR; its native incremental Householder state is not yet wired in.
- Explicit-context SRC keeps tensor payloads and intermediates CUDA-resident; only bounded rank and error decision metadata returns to the host.
- The chain path was cross-checked against the paper and authors' reference code; the tree recurrence was separately derived and provenance-audited.

TreeACI: alternating cross interpolation on trees

tensor4all-rs

- ACI approximates pointwise functions without forming the exact high-rank result (Ritter et al.). The chain crate ports AlternatingCrossInterpolation.jl.
- **New:** native ACI on compatible TreeTNs, with no chain conversion or full-tensor materialization, and n-way products in one run without intermediate products.
- Same core controls as chain ACI, plus tree traversal and explicit resource / cache budgets.
- A full round visits every directed edge once: a minimum-retracing forward walk of length $2|E| - \text{diameter}$ followed by the reverse diameter spine. A chain gives the familiar forward / backward pair.

- **PartitionedTreeTN**: adaptive patching (Grosso et al.) generalized from tensor trains to arbitrary tree topologies.
- Patch error budgets proportional to patch volume, $\varepsilon_p^2 \propto \text{vol}_p$.
- **Reconstruction**: repartition a patched target under a global L2 allowance; operators on index subsets such as the quantics Fourier transform.
- **PartitionedTT adaptive interpolation**: patch waves scheduled by hataori-rs, deterministic sampling across serial, Rayon and MPI runs.

hataori-rs: tasks over threads and ranks

hataori-rs

- A Rust engine for a dynamically scheduled `pmap`, inspired by `Distributed.jl`. Task granularity: seconds to minutes.
- One API, four modes chosen by Cargo features: serial, Rayon, MPI, hybrid (ranks × threads).
- MPI through `rsmpi` or a runtime-loaded MPI ABI, so it shares the runtime of `MPI.jl` and `mpi4py`.
- Used by `tensor4all-rs` adaptive interpolation and a `tenferro` adapter. A long-lived distributed runtime is under design.

Bindings: Tensor4all.jl and Python

bindings

Tensor4all.jl (best effort)

- For light use and teaching; heavy users call tensor4all-rs directly
- Windows CI, RustToolChain update, QR tolerance through the FFI
- Usage audit skill: 12 downstream failure modes; batched readout beat pointwise readout by three orders of magnitude

tensor4all-py (PyO3 prototype)

- Direct PyO3 boundary, not through the C API
- Index, Tensor, TreeTN, batched TreeTCI and quantics cross interpolation from NumPy callbacks
- Not yet: workspace membership, CI, wheels

How we develop, and what comes next

Process

- Issues carry acceptance criteria; agents implement, read-only reviewer models gate design and diff
- **Profile first:** no optimization without a paired measurement
- Provenance policy and CITATION.cff per repository

Next steps

- PartitionedTreeTN: adaptive TCI with region splitting and hataori-rs parallel scheduling
- SRC and TreeACI benchmarks against zip-up and fitting on quantics workloads
- Python: CI, wheels; Tensor4all.jl: catch up to TreeTN

Discussion: which of these do you need first?

References

1. Y. Núñez Fernández, M. K. Ritter, et al., Learning tensor networks with tensor cross interpolation: new algorithms and libraries, SciPost Phys. 18, 104 (2025).
2. Y. Núñez Fernández et al., Learning Feynman diagrams with tensor trains, Phys. Rev. X 12, 041018 (2022).
3. M. K. Ritter, Y. Núñez Fernández, M. Wallerberger, J. von Delft, H. Shinaoka, X. Waintal, Quantics tensor cross interpolation, Phys. Rev. Lett. 132, 056501 (2024).
4. M. K. Ritter et al., Alternating cross interpolation, arXiv:2604.00037 (2026); AlternatingCrossInterpolation.jl.
5. C. Camaño, E. N. Epperly, J. A. Tropp, Successive randomized compression, arXiv:2504.06475 (2025); RandomMPOMPS.
6. G. Grosso, M. K. Ritter, et al., Adaptive patching for tensor train computations, arXiv:2602.22372 (2026).
7. M. Fishman, S. R. White, E. M. Stoudenmire, The ITensor software library, SciPost Phys. Codebases 4 (2022); ITensorNetworks.jl.
8. S. R. White, Phys. Rev. Lett. 69, 2863 (1992); J. Haegeman et al., Phys. Rev. B 94, 165116 (2016); M. Yang and S. R. White, Phys. Rev. B 102, 094315 (2020).
9. J. Chen and M. Lindsey, Direct interpolative construction of the DFT as an MPO, arXiv:2404.03182.
10. Software: github.com/tensor4all/tenferro-rs, [tensor4all-rs](https://github.com/tensor4all-rs), [tenferro-benchmark](https://github.com/tenferro-benchmark), [Tensor4all.jl](https://github.com/Tensor4all.jl); github.com/shinaoka/hataori-rs.